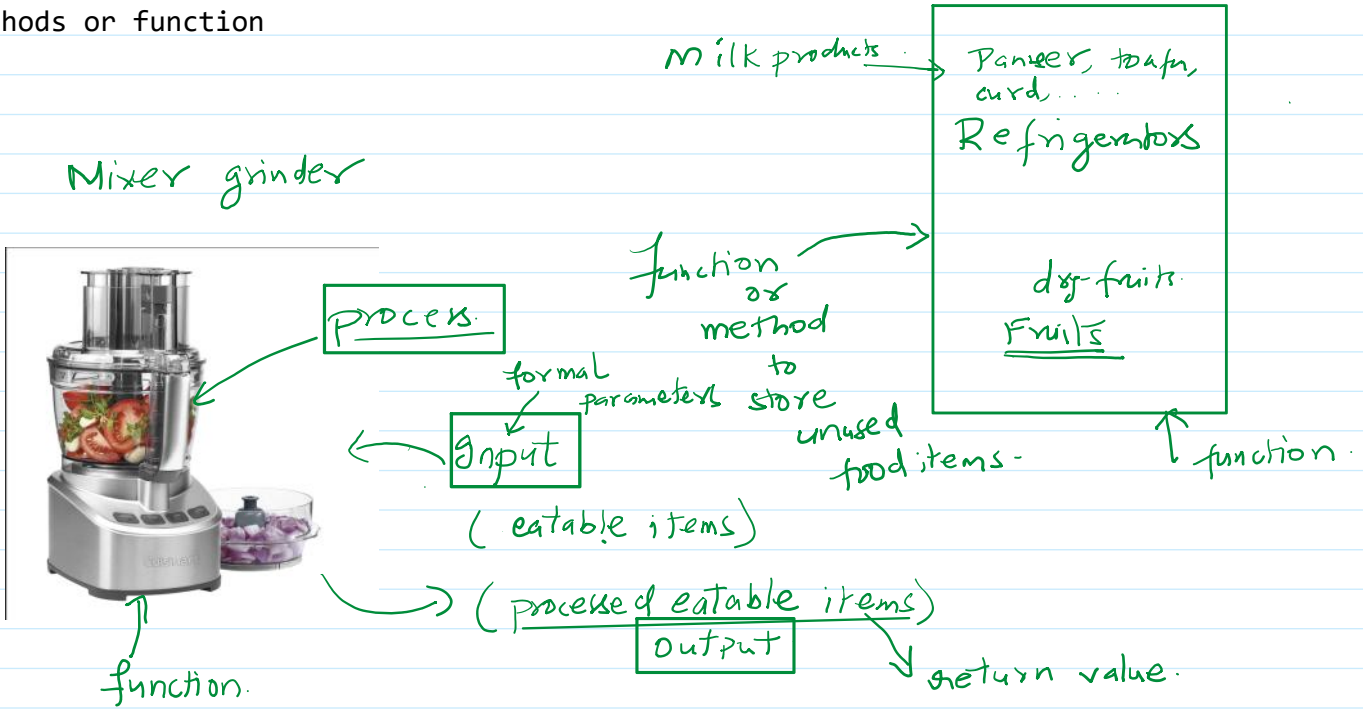
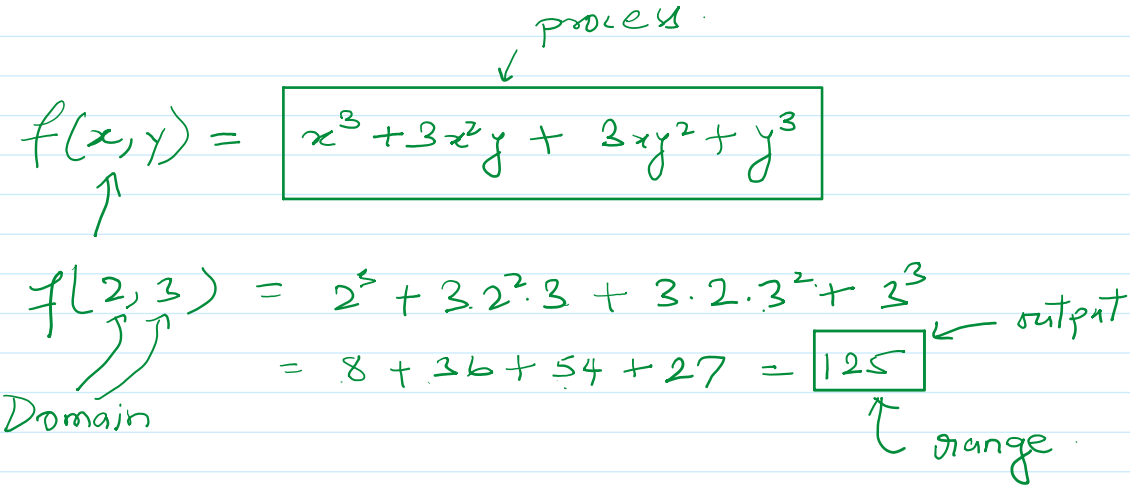
Methods in Java

17 January 2026 10:31

Methods or function



(Palak Paneer)
needs to be grind
cut.



User-defined Methods in Java

A user defined method is a method created by the programmer to perform specific task.

Java provides many built-in methods (like, System.out.println(), Math.sqrt(), Math.max(), Math.abs() etc)

Java provides many built-in methods (like, System.out.println(), System.out.print(), Math.sqrt(), Math.pow(), Math.cbrt() etc), but when these methods are not sufficient, we define our own methods to modularize and reuse code.

Why use user-defined methods?

User defined methods help to:

- **Improves code reusability.**
- **Make programs modular.**
- **Increase readability.**
- **Simplify debugging and maintenance.**
- **Reduce code duplication.**

```
double a = Math.sqrt(16);   // 4.0
```
4.0 ← output; Math.sqrt(16) ← Method call; 16 ← input

Syntax of a User-defined method:

```
<accessSpecifier> <accessModifiers> returnType methodName(<parameters>)
{
   //method body
   <return value>; // if return type is void then it is optional
}
```

AccessModifier	**Static or non-static**
AccessSpecifier	**public, private, protected or default**
Return type	**Data type returned(int, float, double, not returning any data then it is void, etc)**
methodName	**Name of method**
Return	**Sends value back to caller**

accessSpecifier, accessModifier, parameter

cbrt

```
public static double cbrt(double a)
```

Returns the cube root of a double value. For positive finite x, cbrt(-x) == -cbrt(x); that is, the cube root of a negative value is the negative of the cube root of that value's magnitude. Special cases:

- If the argument is NaN, then the result is NaN.
- If the argument is infinite, then the result is an infinity with the same sign as the argument.
- If the argument is zero, then the result is a zero with the same sign as the argument.

The computed result must be within 1 ulp of the exact result.

Parameters:

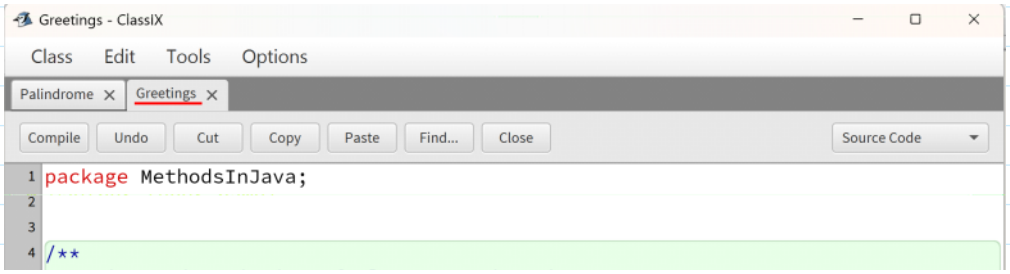
a - a value.

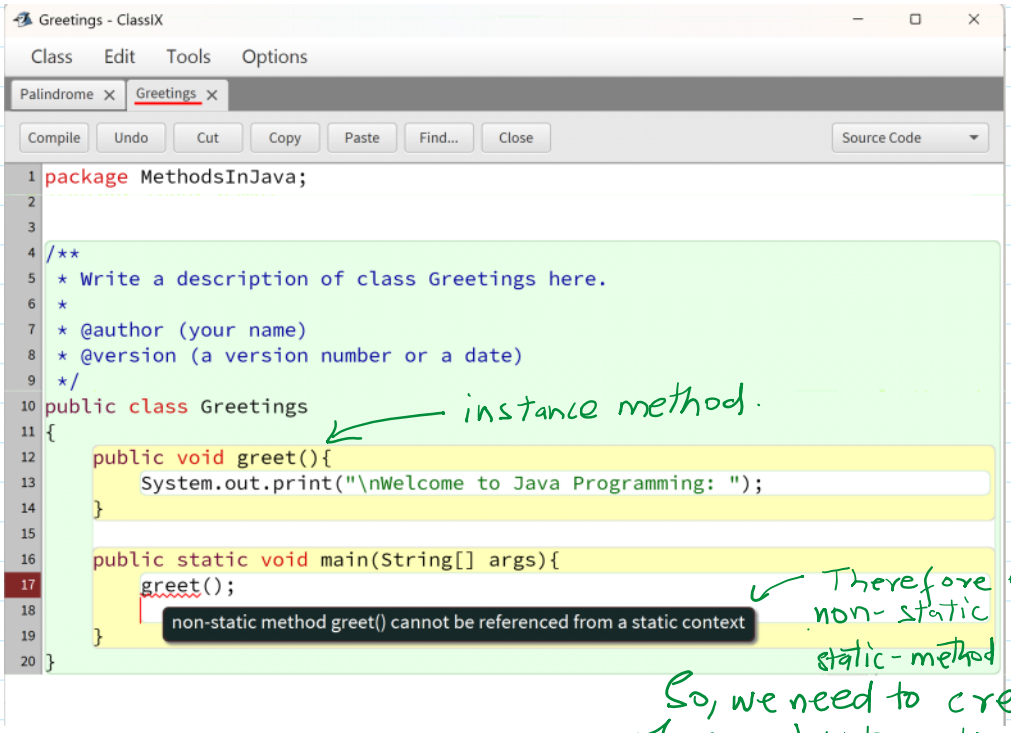
Returns:

the cube root of a.

Since:

1.5





instance method.

Therefore we cannot call, non-static method from static-method directly.

So, we need to create object or instance of the class to call the method.

```
public class Greetings
{
    public void greet(){ //function prototype
        System.out.print("\nWelcome to Java Programming: ");
    }

    public static void main(String[] args){
        Greetings g = new Greetings();
        g.greet(); //function signature is used to call the function
    }
}
```

```
package MethodsInJava;
import java.util.*;


/**
 * Write a description of class Greetings here.
 *
 * @author (your name)
 * @version (a version number or a date)
 */
public class Greetings
{
    public void greet(){ //function prototype
        System.out.print("\nWelcome to Java Programming: ");
    }
    //function overloading the name of function is same but differs
    // with number of parameters or type of parameters.
    public void greet(String name){ //function prototype
```

```
        System.out.print("\n"+name+"! Welcome to Java Programming! " );
    }

    public static void main(String[] args){
        System.out.print("\f");
        Greetings g = new Greetings();
        g.greet(); //function signature is used to call the function

        Scanner sc = new Scanner(System.in);
        System.out.print("\nEnter your name: ");
        String name = sc.nextLine();
        g.greet(name);
    }
}
```

Method is a code snippet use to solve a particular part of a problem.

Advantages of using Methods

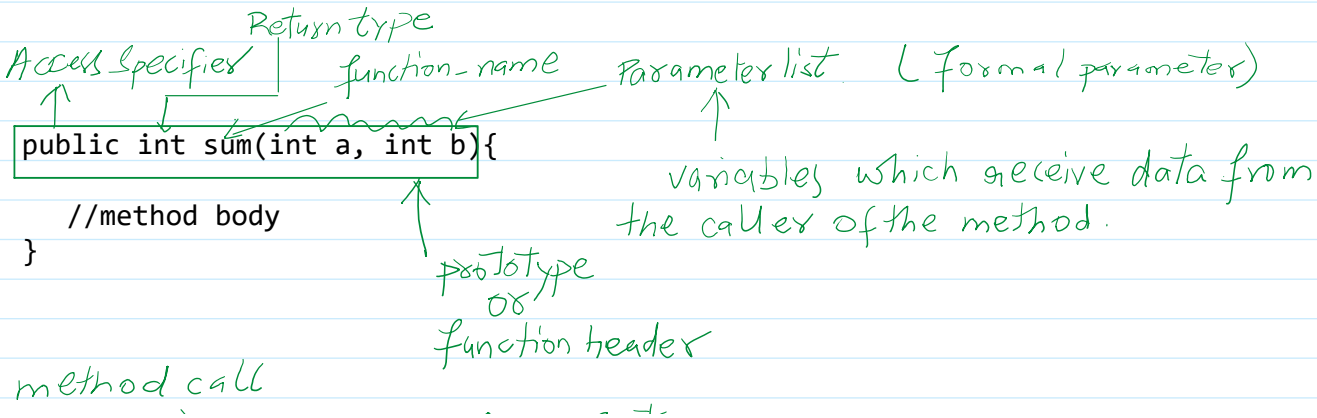
- **Reusability:** Methods provide code reusability. Once a method is defined, it can be called multiple times from various parts of the program.
- **Modularity:** Methods help breakdown complex problems into smaller, manageable pieces.
- **Maintainability:** With methods, updating and debugging code becomes easier, since changes in one part of the program do not necessarily impact other parts.
- **Abstraction:** Methods provide a way to encapsulate implementation details, exposing only the necessary parts to the users.

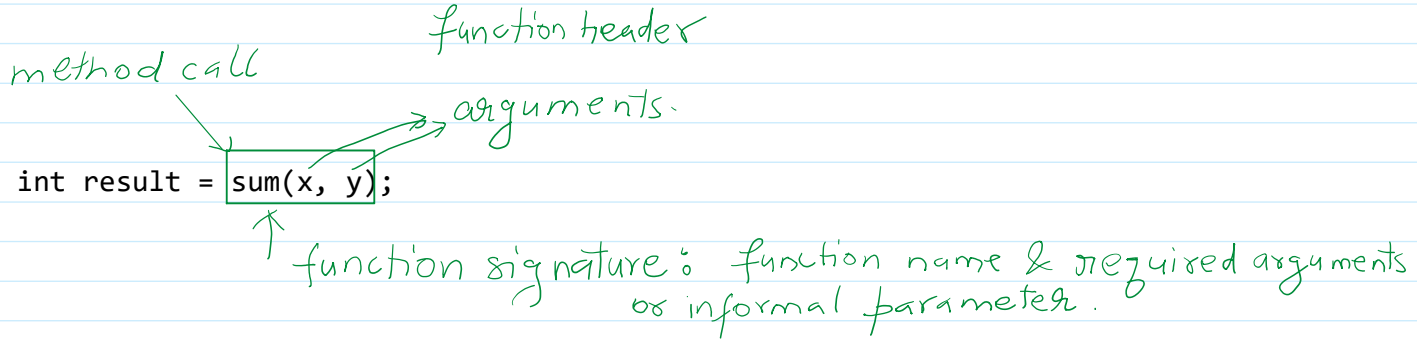
Method Naming Rules

While defining a method, remember that the method name must be a verb and start with a lower case letter. If the method name has more than two words, the first name must be a verb followed by an adjective or noun. In the multi word method name, the first letter of each word must be in uppercase except the first word. For example:

Single-word Method Name: sum(), area()
Multi-word Method Name: areaOfCircle(), endsWith(), indexOf(), lastIndexOf()

It is also possible that a method has same name as another method in the same class. It is known as method overloading.





Access Specifier Access specifier or modifier is the access type of method. It specifies the visibility of the method. Java provides four types of access specifier:

- **public:** The method is accessible to all classes when we use the public specifier in our application.
- **private:** When we use a private access specifier, the method is accessible only in the classes in which it is defined.
- **protected:** When we use a protected access specifier, the method is accessible within the same package or subclasses in a different package.
- **default:** When we do not use any access specifier in the method declaration, Java uses the default access specifier by default. It is visible only from the same package.

Return type: Return type is a data type that the method returns. It may have a primitive data type, object, Collection, void, etc. If the method does not return anything, we use the void keyword.

Types of Methods

There are two types of methods in Java:

- **Predefined method:** The methods that are already defined in Java class libraries are known as predefined methods. It is also known as the standard library method or built in method. We can directly use these methods just by calling them in the program at any point.
  For example: Math.pow(), Math.sqrt(), length(), substring(), etc.

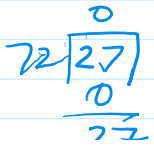
- **User-Defined Method:** The method written by the user or programmer is known as a user defined method. These methods can be modified or customized according to the requirements.

  Calling User defined Method

  In Java, calling or invoking a method depends on whether the method is static or non-static.
  If the method is static, we can call it directly using the class name or from within the same class. For example, Math.pow(), here Math is a class and pow() is static function. Note that we need not to create an object for calling static methods.

  If the method is non-static, we must create an object of the class and use it to call the method.

```
public int HCF(int x, int y){
        for(int r = 0; y > 0; r = x % y, x = y, y = r );
```



```
public int HCF(int x, int y){
        for(int r = 0; y > 0; r = x % y, x = y, y = r );
        return x;
    }
```

72|27|
0
27

x = 27 72 27 18 9
y = 72 27 18 9 0
r = 0 27 18 9 0

Call by Value and Call by Reference in Java

Call by Value: In Java, when a method is called by passing a value as an argument, it is referred to as **call by value**. With Javas call by value feature, a copy of the variable is passed to the method. As a result, any modification made within the method only effect that method and do not impact the original variable in the main method.
This is the primary distinction between Javas call by reference and call by value functions. Passing a variable with basic data type is considered a call by value. Therefore, any modification made to the variable will not be reflected in the caller's scope.

Important points to remember

- Java always uses call by value, even for objects.
- When a primitive data type (like int, double, float) is passed to a method, a copy of the value is made.
- Changes inside the method do not affect the original variable.

Example 1: Call by Value with Primitive Data

```
package MethodsInJava;
import java.util.*;

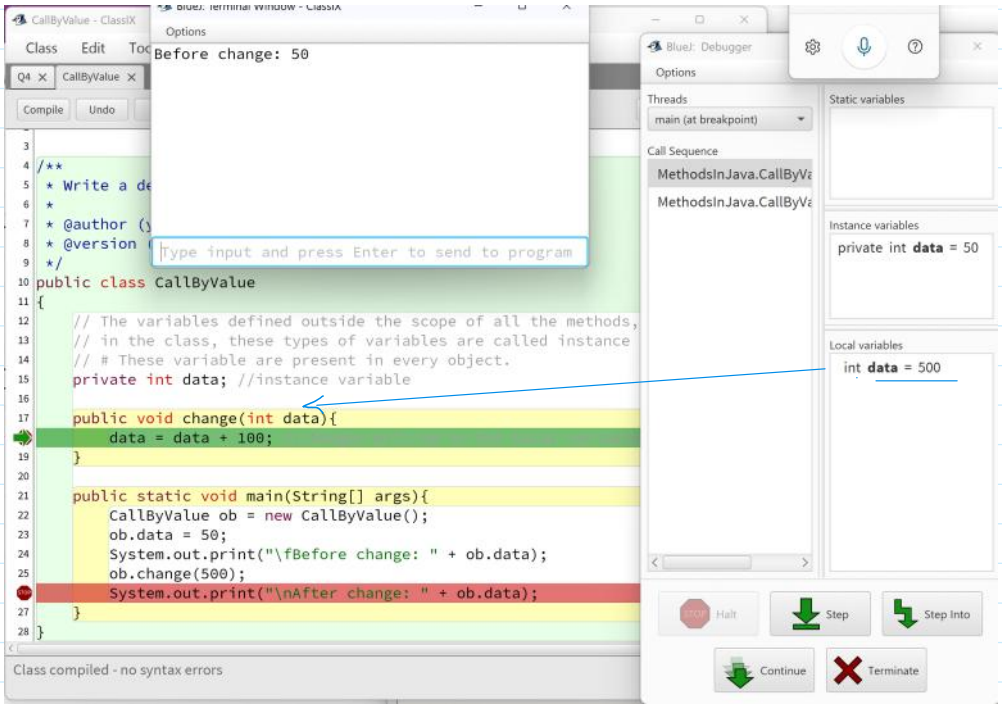

/**
 * Write a description of class Q4 here.
 *
 * @author (your name)
 * @version (a version number or a date)
 */
public class Q4
{
    public int fact(int n){
        int f = 1;
        for(;n > 1; f *= n, n--);
        return f;
    }

    public static void main(String[] args){
        int n, m;
        Scanner sc = new Scanner(System.in);
        System.out.print("\fEnter any positive integer for n: ");
        n = sc.nextInt();
        if(n < 1){
            System.out.print("\nInvalid input: ");
```

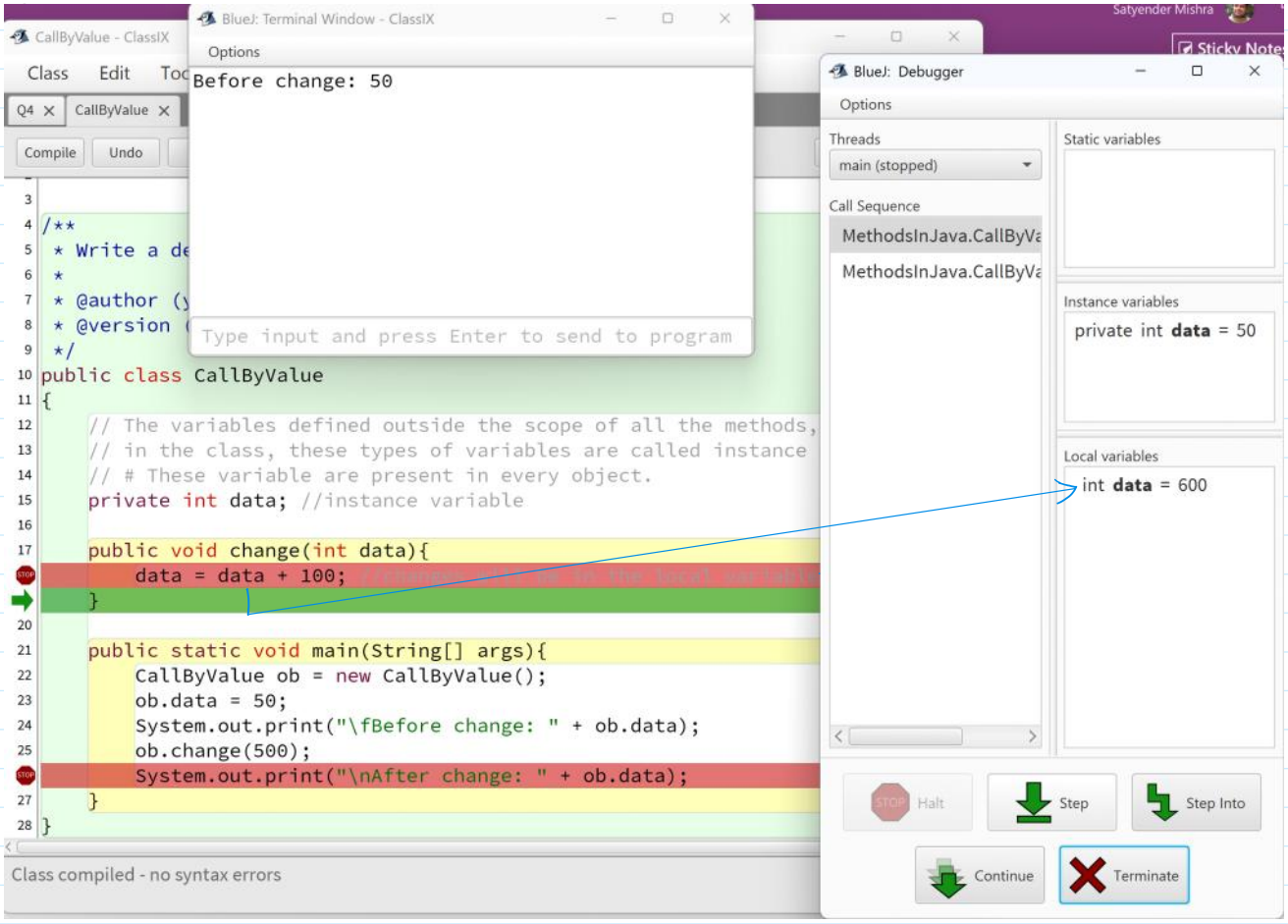
call by value
copy
local to fact

```
            n = sc.nextInt();
            if(n < 1){
                System.out.print("\nInvalid input: ");
                return; //exit from main() method
            }
            System.out.print("Enter any positive integer for m: ");
            m = sc.nextInt();
            if(m < 1 ){
                System.out.print("\nInvalid input: ");
                return; //exit from main() method
            }
            if((n - m) < 1){
                System.out.print("\nValue of " + n + " should be greater than
"+m);
                return; //exit from main
            }
            Q4 ob = new Q4();
            int S = ob.fact(n)/(ob.fact(m) * ob.fact(n-m));
            System.out.print("\nS = " + S);
        }
    }
```

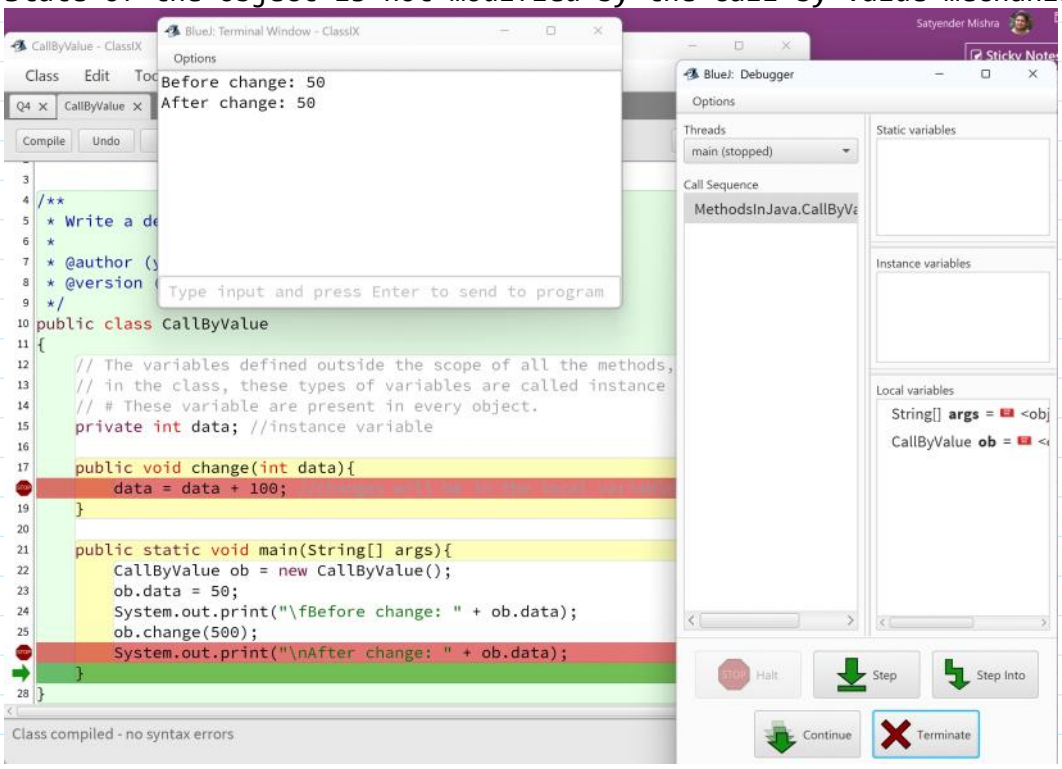
Another example:



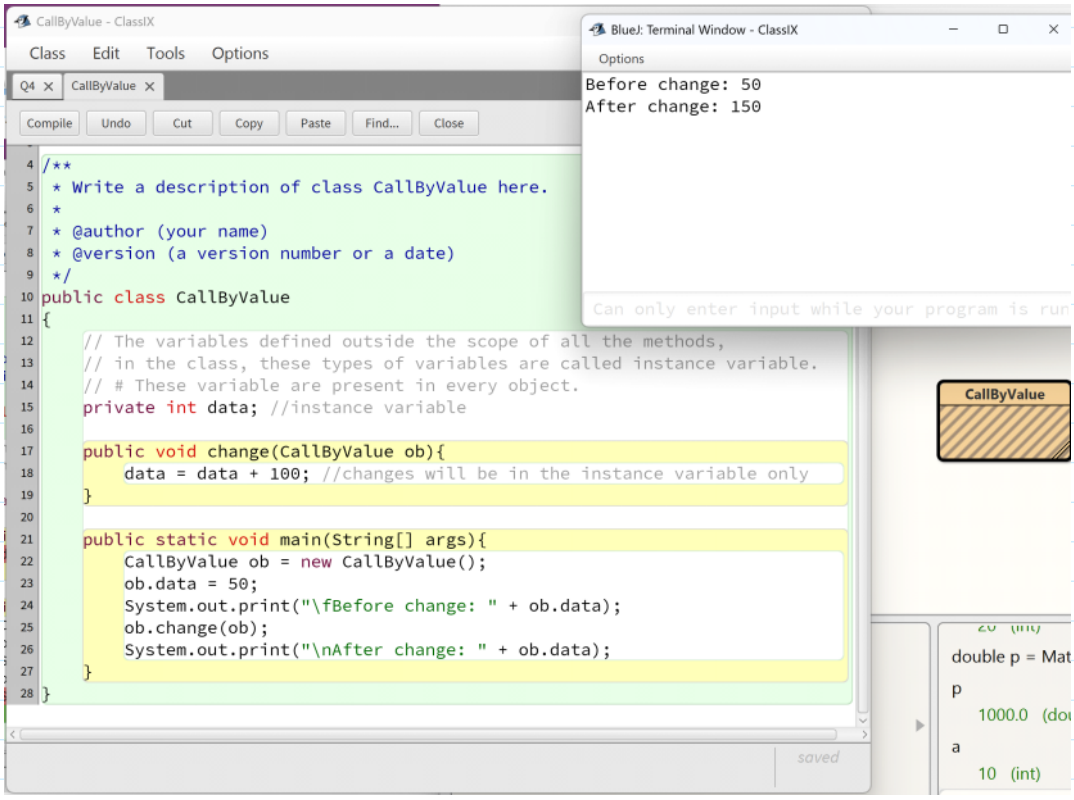
Updated the local variable



State of the object is not modified by the call-by-value mechanism



Call by Value with Object Reference



Call by Reference (Object Reference)

In Java, **call-by-reference** means passing the reference (memory address) of the object by value to a method. Even though Java strictly uses **call-by-value**, it duplicates the reference before passing it as a value to the method when we send the reference of an object. The most important difference between call by value and call by reference in Java is that the copy the reference also points to the same address, ensuring that all modifications are reflected in the main method.

Important Points to Remember

- Java **does not** support true call by reference. It passes the reference by value.
- When an object is passed, a **copy of the reference** is made.
- The reference still points to the same objects, so modification **inside the method** affect the original object.

Difference between call by value and call by reference

Feature	Call by value	Call by reference
Definition	A copy of the actual value is passed to the method.	A copy of the reference (address) is passed to the method.
Java Support	Java uses call by value for both primitive types and objects.	Java does not support true call by reference. Instead it passes the reference by value.
Effect on Original Data	Changes made to the value inside the method do not affect the original variable.	Changes to the object's state within the method impact the original object (but not the reference itself).

Objects	A copy of reference to the object is passed. Changes to the objects attribute affect the original object.	Not applicable. Java does not support true call by reference.
Example with Objects	If an object is passed, modifying its field. Will affect the original object, but reassigning the reference will not.	Not applicable
Examples with primitive	If an int is passed, modifying it inside the method does not impact the original int.	Not applicable
Memory consumption	It requires more memory because a copy of the variable is created.	The reference points to the same memory location as the original object, so no more memory is needed. Meaning it's more memory efficient.
True call by Reference?	Yes	No

Method overloading

Method overloading is one of the important features of **Polymorphism** in Java.

Poly(means, multiple) morph(means, forms) of same type. For example, apple may be red or green, therefore apple is polymorphic.

It allows a class to have multiple methods with the same name but with **different parameter list.**

Definition:

Method overloading means:

Defining more than one method with the same name in the same class but changing:

- Number of parameters.
- Type of parameters.
- Order of parameters.

For example:

```
int add(int x, int y)
float add(int x, float y) //type is changed
float add(float x, int y) //ordering of parameters changed
int add(int x, int y, int z) //Number of parameters
```

Why use method overloading?

Method overloading helps to:

- Improve code readability.
- Reuse the same method name for similar tasks.
- To reduce complexity.
- Achieve compile time polymorphism.

For example:

```
package MethodsInJava.Overloading;


/**
 * Write a description of class ArithmeticOperations here.
 *
 * @author (your name)
 * @version (a version number or a date)
 */
public class Arithmetic
{
    public static int add(int x, int y){
        return (x + y);
    }
    public static float add(int x, float y){
        return (x + y);
    }
    public static float add(float x, int y){
        return (x + y);
    }
    public static float add(float x, float y){
        return (x + y);
    }
    public static double add(double x, double y){
        return (x + y);
    }


}
```

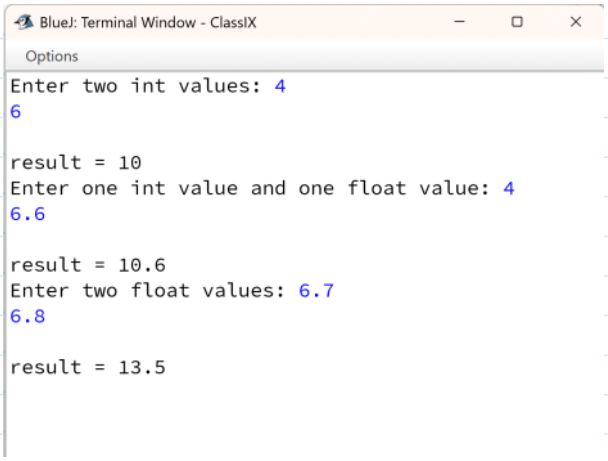
```
package NestedLoops;
import java.util.*;
import MethodsInJava.Overloading.Arithmetic;

/**
 * Write a description of class Mathematics here.
 *
 * @author (your name)
 * @version (a version number or a date)
 */
public class Mathematics
{
    public static void main(String[] args){
        Scanner sc = new Scanner(System.in);
        int x, y, r;
        float f1, f2, fr;
        double d1, d2, dr;
        System.out.print("\fEnter two int values: ");
        x = sc.nextInt();
        y = sc.nextInt();
```

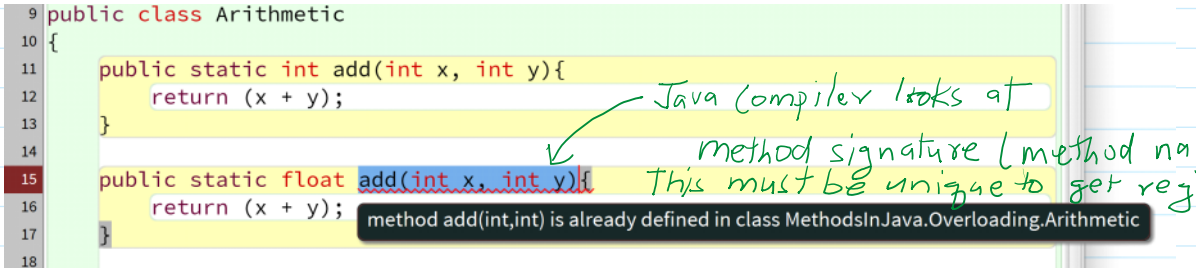
```
        r = Arithmetic.add(x, y);
        System.out.print("\nresult = " + r);
        System.out.print("\nEnter one int value and one float value: ");
        x = sc.nextInt();
        f1 = sc.nextFloat();
        fr = Arithmetic.add(x, f1);
        System.out.print("\nresult = " + fr);
        System.out.print("\nEnter two float values: ");
        f1 = sc.nextFloat();
        f2 = sc.nextFloat();
        fr = Arithmetic.add(f1, f2);
        System.out.print("\nresult = " + fr);

    }
}
```

Output:



int add(int x, int y)
float add(int x, int y) //this is wrong, compiler won't recognize this as different function



Important Note: Method overloading cannot be achieved by changing only the return type. This causes compilation error.

Compile-time Polymorphism
Method overloading is called compile-time polymorphism.

Key points to remember:
1. Same method name is required.

2. Parameters must differ.
3. Return type alone cannot overload methods.
4. Overloading improves readability.
5. Constructors can also be overloaded(I will teach you during class and objects).